From StableHLO to Tensix

libtt as a hermetic, full-stack runtime for Tenstorrent accelerators

libtt technical report

15 July 2026

Contents

Abstract

libtt packages the Tenstorrent JAX execution stack as one PJRT shared library. Its pinned TT-XLA frontend, TT-MLIR compiler, TTNN runtime, TT-Metal host code, device kernels, and runtime assets are built together and linked into `libtt.so`; the target system does not need a separate compiler or TT-Metal installation. This report describes that architecture, relates it to Google's `libtpu.so` deployment model, and maps libtt's optimizations onto the StableHLO, TTIR, TTNN, D2M, TTKernel, TTMetal, runtime, and kernel levels of the stack. The organization follows patches and optimization concepts, rather than Git commits, because the important changes are vertical slices: a SwiGLU epilogue, for example, begins as graph recognition, becomes a TTNN operation and serialized attribute, selects a runtime program factory, and ends as a Dst-resident Tensix kernel. On Qwen3-8B decode on a Blackhole P150, the established optimization line raises end-to-end throughput from 16.265 to 26.123 tokens/s (+60.61%). A newer 110-core down-projection specialization adds 5.96% pure decode throughput in a same-build 32-sample A/B test, reaching 27.753 tokens/s and reducing the projection kernel from 217.188 to 154.932 microseconds per layer. In contrast, widening the SwiGLU K block from two to four tiles changes throughput by only +0.27% ($p = 0.45$). The results show why an open, centrally built stack is useful: profitable inference work crosses graph semantics, layout policy, runtime dispatch, data movement, and device arithmetic, and those layers must be changed and measured together.

Executive summary

libtt is a self-contained Tenstorrent backend for JAX. Its public deployment surface is a single file, `libtt.so`, loaded through PJRT. Internally that file contains the TT-XLA PJRT implementation, the TT-MLIR compiler, the TTNN and TT-Metal runtime code, and a compressed archive of the TT-Metal device-runtime assets. The archive is materialized automatically on first load. This resembles the role of Google's `libtpu.so`, which combines the TPU compiler, driver, and hardware communication logic behind a shared-library boundary [1]. The analogy is architectural, not an assertion that the implementations are related.

The second design choice is equally important: the complete open stack is assembled by one Bazel build. libtt pins upstream revisions, supplies Bazel overlays for projects that are not otherwise part of the same build graph, and applies a visible patch series at fetch time. The result is close to the hermetic-build ideal described by Bazel—tools and dependencies are explicit inputs rather than assumptions about the host [2]. For performance engineering, this gives a human or coding agent one place to change every layer from compiler pattern matching to device dataflow, rebuild one artifact, and run one serving benchmark.

The measured work supports six conclusions:

1. **Recovering model semantics is foundational.** JAX frequently presents RMSNorm and SiLU as primitive arithmetic. TTIR and TTNN patterns recover those operations, expose Qwen's QKV structure, and preserve cache dtypes. The decode-foundation patch group contributes +22.58%; expanded-SiLU recognition adds +3.30%; and QKV projection fusion adds +1.15%.
2. **Layout policy can dominate a small kernel.** Runtime width-sharding of one-token RMSNorm across 16 cores adds +13.61%, the largest isolated gain in the established sequence.
3. **Fusion is valuable only at the right materialization boundary.** Combining the gate/up matmuls alone regresses by 1.69%, and fusing SiLU with a still separate multiply is inconclusive at -0.65%. A true producer-side matmul-SwiGLU epilogue keeps gate/up tiles in Dst and adds +2.73%.
4. **More blocking is not automatically better.** Increasing the fused SwiGLU K block from two to four tiles yields +0.27% in the 32-sample test ($p = 0.45$). Widths 4, 8, and 16 remain experimental; two stays the default.
5. **Core geometry remains a major decode lever.** A specialized 110-core down projection replaces a 64-core program, raises effective BFP8 weight bandwidth from 246.2 to 345.2 GB/s, lowers kernel time by 28.7%, and improves pure decode throughput by 5.96%.
6. **Trace capture matters outside the token loop.** Recording the fixed-shape one-tile prefill graph reduces mean time to first token from 618.84 to 59.15 ms. The primary 128-token benchmark improves by 10.23%, while a direct trace-only A/B test finds no significant decode change.

The established main snapshot reaches 26.123 tokens/s, up 60.61% from the documented baseline. Tenstorrent's tt-inference-server v0.10.0 reference reaches 24.148 tokens/s on the same P150, model, prompt, output length, and serial cache-disabled workload. The established libtt result is 8.18% higher, subject to a timing-scope caveat: the reference uses loopback-client time while the

original libtt sequence uses server-reported request time. The newer down-projection experiment uses a streaming client clock and is therefore reported separately, not appended to the older cumulative series.

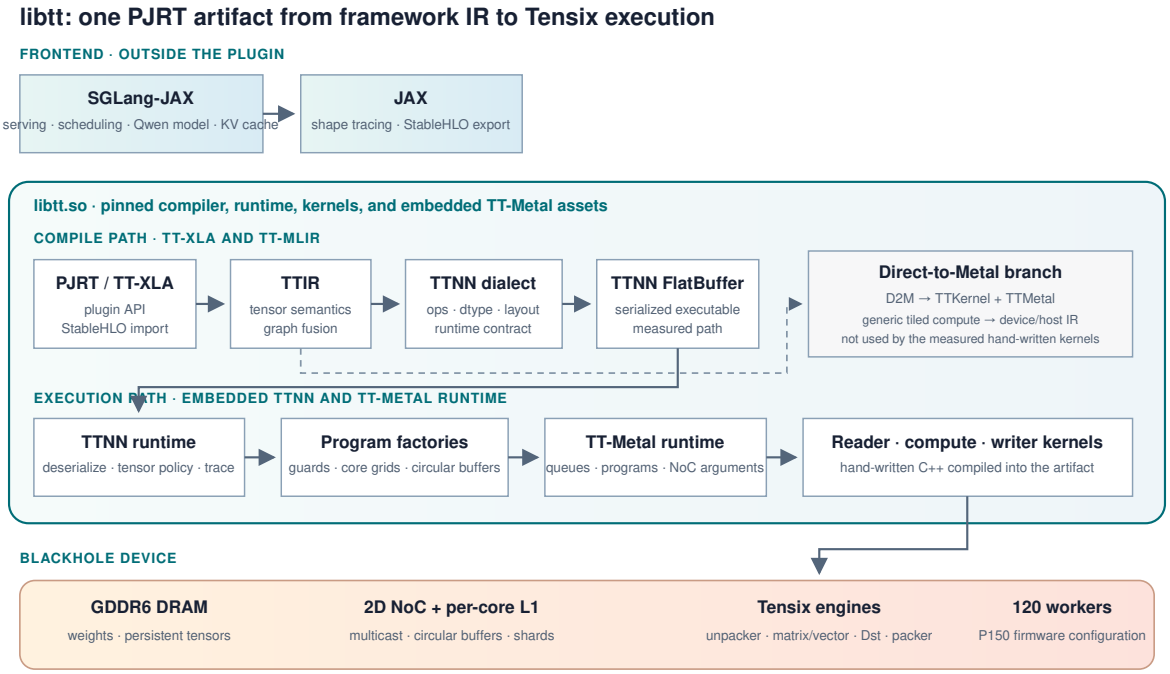

Figure 1: libtt's serving, compilation, runtime, and device layers.

1 Purpose and design philosophy

1.1 What libtt is

PJRT defines a uniform device interface through which frameworks call opaque, device-specific plugins [3]. libtt implements that boundary for Tenstorrent:

```
JAX process
  `-- dynamically loads libtt.so through PJRT
       |-- compiles StableHLO for Tenstorrent
       |-- allocates and transfers device buffers
       |-- loads and executes serialized TTNN programs
       `-- initializes the embedded TT-Metal runtime assets
```

The repository's own C++ is intentionally small. One library extracts the embedded runtime archive to a fingerprinted temporary directory and sets `TT_METAL_RUNTIME_ROOT`; an always-linked constructor performs that setup before PJRT initialization; and the final shared-library rule links the upstream plugin while exporting only the PJRT entry points. Most functionality comes from pinned open-source dependencies, Bazel overlays, and libtt's patch series.

The deployment contract is therefore stronger than "a plugin that finds an installed SDK." The TT compiler and TT-Metal runtime do not need to be installed on the target. Given a compatible driver/firmware environment and `libtt.so`, the software stack needed to compile and run a JAX program travels with the plugin. This is the specific sense in which libtt follows the `libtpu.so` model [1].

1.2 One build graph for the complete open stack

libtt uses Bazel modules and repository rules to pin TT-UMD, SFPI, TT-Metal, LLVM, StableHLO, TT-XLA, TT-MLIR, Shardy, and supporting C++ libraries. Bazel overlays define targets where upstream projects use another build system; patch lists adapt and optimize those sources before compilation. The `//:tt` target then produces `bazel-bin/libtt.so`.

This organization has three practical effects:

- **Reproducible scope.** A source revision, overlay, patch, compiler toolchain, and link rule are all reviewable inputs to the same build.
- **Atomic cross-layer changes.** A compiler rewrite, FlatBuffer schema update, runtime handler, TTNN program factory, and Metal kernel can be built and tested as one change instead of coordinating installed components.
- **Agent-friendly iteration.** An agent can inspect the model graph, compiler pass, runtime dispatch, and kernel code in one workspace; modify the smallest appropriate abstraction; rebuild one target; and run the README benchmark. The important property is not automation alone but the absence of opaque or separately versioned layers between diagnosis and implementation.

The open-source nature of TT-XLA, TT-MLIR, TTNN, TT-Metal, and the surrounding toolchain is what makes the last point technically meaningful. Tenstorrent's own compiler overview describes a similarly open chain from framework frontends through TT-MLIR dialects to TT-Metalium [6, 7]. libtt turns a pinned slice of that chain into one distributable artifact.

1.3 Cross-cutting optimization as the design test

The matmul-SwiGLU epilogue is the clearest test of the architecture. No single layer can implement it correctly:

1. A graph pass must prove that a paired gate/up matmul feeds the exact `silu(gate) * up` expression.
2. TTNN IR needs an operation-level representation that preserves the fused semantic through lowering.
3. The serialized executable needs to carry the new matmul attribute.
4. The runtime must dispatch that operation into TTNN.
5. TTNN must select a program factory only for supported shapes, dtypes, layouts, and hardware.
6. Reader, compute, and writer kernels must keep the paired tiles resident in Dst and emit only the half-width final result.

libtt makes those six steps one patch concept, even though they touch two upstream repositories and several abstraction levels. Organizing this report by that concept is more informative than organizing it by the commits used to collect intermediate measurements.

2 Compilation and execution architecture

2.1 The framework boundary: JAX, StableHLO, and PJRT

SGLang-JAX owns HTTP serving, tokenization, scheduling, the JAX Qwen model, and the paged KV cache [15]. JAX traces each shape-specialized computation and hands the plugin StableHLO. StableHLO is a portable high-level operation set between ML frameworks and compilers [4]; PJRT is the API boundary that keeps the device-specific compiler and runtime opaque to JAX [3].

The steady-state request path is:

```
SGLang-JAX → JAX trace → StableHLO → libtt/PJRT
           → TT-XLA → TT-MLIR → TTNN executable
           → TTNN runtime → TT-Metal programs → Blackhole
```

Compilation and execution are both behind `libtt.so`. StableHLO is the incoming contract; a serialized TTNN program is the principal executable artifact in the measured path.

2.2 Why TT-MLIR has several IRs

MLIR is designed around extensible dialects at different abstraction levels [5]. TT-MLIR uses that capability to represent model semantics, layouts, generic tiled computation, device kernels, and host/device orchestration at different points in lowering [7]. The distinctions matter because an optimization should live at the highest level that still contains the information it needs.

Table 2.1: The TT-MLIR IR ladder and where libtt's measured path uses it.

Representation	Abstraction and purpose	Relationship to current libtt optimizations
StableHLO	Framework-portable tensor graph with specified operation semantics [4].	Input to TT-XLA/TT-MLIR. Framework algebra such as expanded SiLU is visible here and after import, but libtt's patches generally match it in TTIR/TTNN passes.

Representation	Abstraction and purpose	Relationship to current libtt optimizations
TTIR	Hardware-aware, high-level tensor IR. It preserves named tensor operations while permitting fusion and canonicalization before a concrete runtime API is chosen [7].	JAX RMSNorm recognition, expanded-SiLU recovery, shared-LHS matmul grouping, role propagation, and part of QKV/RoPE fusion.
TTNN dialect	High-level tensor IR designed to model the TTNN library closely. Its types and attributes carry tiled layouts, memory spaces, sharding, and device data types [7, 8].	QKV composite fusion, `matmul_swiglu`, layout selection, cache return types, and lowering to the serialized TTNN operation graph.
TTNN FlatBuffer	Serialized executable operation graph consumed by the TTNN runtime. It is an artifact rather than an MLIR dialect.	Carries operation parameters such as the fused-SwiGLU matmul flag across the compiler/runtime boundary.
D2M dialect	Generic tensor/memref computation analogous to `linalg.generic`, augmented with grids, sharded tensors, circular buffers, and explicit data movement [7].	An alternative direct-to-metal route. The measured custom SwiGLU and down-projection kernels are hand-written TT-Metal/TTNN patches, so they do **not** pass through D2M.
TTKernel dialect	Low-level device-kernel IR exposing circular buffers, tile registers, NoC transactions, and synchronization with an intended near one-to-one mapping to TT-Metal kernels [7].	Relevant to generated direct-to-metal kernels; not the representation of the hand-written C++ kernels measured here.
TTMetal dialect	Host/device interop IR for allocation, transfers, program creation, and enqueue operations [7].	Part of the direct-to-metal compiler route. The measured TTNN route instead invokes TT-Metal host APIs from the TTNN runtime.

The main lowering branch in this report is therefore:

```
StableHLO → TTIR → TTNN dialect → TTNN FlatBuffer
                                        ↓
                                  TTNN C++ runtime
                                        ↓
```

```
TT-Metal host + C++ kernels
```

TT-MLIR also supports the lower-level branch:

```
TTIR → D2M → TTKernel + TTMetal → generated device/host program
```

Both branches belong in a technical description of TT-MLIR; only the first is the execution path for the benchmarked libtt patches. This distinction avoids calling a C++ program-factory change a "TTKernel IR optimization" when no such IR was involved.

2.3 TTNN runtime and the serialized executable

TTNN IR models operations such as matmul, RMSNorm, SDPA, reshape, and memory configuration. Lowering serializes these into a FlatBuffer. At execution time, the embedded TTNN runtime deserializes the graph, creates tensors, chooses memory configurations, and invokes TTNN operations. A TTNN operation validates its contract and selects a device-operation program factory.

This runtime layer is where dynamic facts can be used without obscuring graph semantics. The decode RMSNorm patch, for example, observes the actual input tensor and creates a 16-core width-sharded memory configuration. The compiler still emits RMSNorm; the runtime chooses the layout specialization.

2.4 TT-Metal programs and Tensix kernels

TT-Metalium exposes a cooperative dataflow model rather than a conventional GPU thread hierarchy [9]. A typical program uses:

- a reader data-movement kernel to fetch tiles from DRAM or another core into L1 circular buffers;
- a compute kernel to unpack tiles, drive matrix/vector engines, and create result tiles; and
- a writer data-movement kernel to drain result buffers to their destination.

Circular buffers are bounded producer/consumer queues shared between the threads of a Tensix core [11]. Readers, compute, and writers can overlap on different tiles, provided that reserve/push/wait/pop and semaphore protocols are correct.

The compute engine exposes SrcA, SrcB, and Dst register sets. Matrix results land in Dst; vector operations can transform them; the packer writes them to an L1 circular buffer [10]. With 16-bit Dst storage and double buffering enabled, the active half contains eight 32-by-32 tiles. That capacity determines the successful SwiGLU geometry: four gate and four up tiles fill Dst exactly.

2.5 Tiles, memory, and low-precision weights

TTNN's standard tile is 32 by 32 elements and contains four 16-by-16 faces [12]. Shapes are padded in their final two dimensions, so logical batch-one decode still presents a full tile row to many kernels. A useful optimization must reason about both logical shape—one valid token—and physical tile shape.

Large weights reside in device DRAM; L1 is smaller, faster, and private to each worker. TTNN `MemoryConfig` values describe interleaved or sharded storage and DRAM or L1 placement. TT-MLIR layout attributes similarly encode how a logical tensor maps to devices, cores, physical shards, memory space, and padding [8].

The benchmark uses BF16 activations and outputs with BFLOAT8_B weights. BFLOAT8_B is block floating point: 16 values share an exponent [13]. It reduces the dominant weight traffic in batch-one decode, with a precision and packing trade-off. The custom matmuls use BF16 packer-L1 accumulation: partial sums are packed into an L1 slot and reloaded for later K blocks, leaving Dst available for the current block's tile group.

2.6 Why autoregressive decode is bandwidth-sensitive

Qwen3 is a decoder-only Transformer family [16]. For a simplified layer,

$$\begin{aligned} u &= \mathrm{RMSNorm}(h), \\ h' &= h + \mathrm{Attention}(Q(u), K(u), V(u); K_{cache}, V_{cache}), \\ v &= \mathrm{RMSNorm}(h'), \\ g &= vW_{gate}, \qquad r = vW_{up}, \\ h_{next} &= h' + (\mathrm{SiLU}(g) \odot r)W_{down}. \end{aligned}$$

RMSNorm and SwiGLU are standard model concepts [17, 18]. During prefill, many prompt rows share each weight read. During serial decode, a projection is logically $1 \times K$ by $K \times N$ for every new token. Weights are reread at each layer and token, arithmetic intensity is low, and launch/materialization overhead becomes visible. This is why the principal wins below expose more width parallelism, reduce DRAM traffic, or remove an intermediate before it is packed from Dst.

3 Optimization map by patch and concept

The table below is the roadmap for the rest of the report. Git revisions remain in the data files for provenance, but the technical unit is the patch concept and its location in the stack.

Table 3.1: Current model-path optimization concepts, implementation levels, and measured effects.

Concept	Principal level / IR	Measured effect
Recover JAX RMSNorm	TTIR graph rewrite	Included in +22.58% foundation bundle
Recognize expanded SiLU	TTIR graph rewrite	+3.30%
Fuse rank-3 decode RoPE	TTIR/TTNN composite resolution	Included in foundation bundle
Fuse Qwen decode QKV structure	TTIR ordering + TTNN fusion	+1.15%
Preserve KV-cache result types	TTNN type inference	Included in foundation bundle
Enable BF8 activation/weight path	Compile options, TTNN lowering, host packing	Foundation/baseline; not isolated
Permit decode layouts	TTNN/TT-Metal validation	Included in foundation bundle
Width-shard decode RMSNorm	TTNN runtime policy	+13.61%
Combine two shared-LHS matmuls	TTIR fusion	-1.69%; enables epilogue
True matmul-SwiGLU epilogue	TTNN IR/FlatBuffer/runtime + TT-Metal	+2.73%
Generalize and remove fallback	TTNN matching/runtime simplification	-0.19%, p = 0.53
Sweep SwiGLU K blocking	TT-Metal program and CB geometry	+0.27%, p = 0.45
Specialize down projection	TTNN program selection + TT-Metal	+5.96% pure decode
Trace one-tile prefill	TTNN/TT-Metal runtime trace	-90.44% TTFT; +10.23% E2E

The corresponding patch index is compact enough to keep the mapping explicit:

```
RMSNorm graph       tt_mlir_fuse_jax_rms_norm.patch
expanded SiLU       tt_mlir_fuse_expanded_silu.patch
decode RoPE         tt_mlir_fuse_rank3_rope_decode.patch
QKV structure       tt_mlir_qwen_decode_qkv_projection_fusion.patch
KV-cache types      tt_mlir_kv_cache_dtype_return_types.patch
BF8 path            tt_mlir_single_chip_activation_dtype_lowering.patch
                    fast_bfloat16_bfp8_pack.patch
decode layouts      sdpa_decode_allow_l1_interleaved_q.patch
                    layernorm_allow_single_core_height_sharded.patch
RMSNorm sharding    tt_mlir_sharded_decode_rms_norm.patch
shared-LHS pair     tt_mlir_fuse_shared_lhs_matmul_pairs.patch
SwiGLU vertical     tt_mlir_fuse_matmul_swiglu.patch
                    matmul_swiglu_epilogue.patch
down projection     down_projection_110_core.patch
```

The +22.58% foundation figure is aggregate because its dependent patches were not isolated. The current blocking and down-projection A/B tests use a streaming decode clock and therefore remain separate from the cumulative sequence.

4 Recovering model semantics in TTIR and TTNN

4.1 RMSNorm recognition

Framework tracing does not guarantee that a named model operation survives as one StableHLO op. JAX RMSNorm can arrive as square, reduce, epsilon addition, reciprocal square root, scaling, and reshapes. The RMSNorm fusion patch proves that structure and replaces it with one semantic normalization operation.

This is the right level for the rewrite. TTIR still exposes tensor algebra and use-def structure, but the replacement can later benefit from TTNN's dedicated normalization operation, layout analysis, and runtime program selection. Matching later in TT-Metal would be impossible because the arithmetic would already be split into independent programs.

RMSNorm recognition is part of the foundation bundle rather than an isolated measurement. It also enables the later runtime-sharding patch: a runtime cannot choose a specialized RMSNorm layout if the compiler never recovered RMSNorm.

4.2 Expanded-SiLU recognition

The observed graph represents

$$\mathrm{SiLU}(x) = x\,\sigma(x) = \frac{x}{1+\exp(-x)}$$

with casts, broadcasts, reshapes, and a splatted scalar one around the core arithmetic. `tt_mlir_fuse_expanded_silu.patch` looks through view-like operations, proves the constant and the shared input, and replaces the expanded tree with a SiLU operation. It removes elementwise launches and intermediates without relying on fragile source-level names. The isolated improvement is **+3.30%**.

4.3 Rank-3 RoPE and QKV projection structure

Decode Q, K, and V are not three unrelated slices. Qwen projects them together, applies per-head RMSNorm to Q and K, reshapes into query and KV head counts, and then applies rotary position embedding. Two patch concepts recover this shape:

- `tt_mlir_fuse_rank3_rope_decode.patch` accepts JAX's rank-3 decode form, temporarily maps it to the existing rank-4 composite, and restores the expected result shape.
- `tt_mlir_qwen_decode_qkv_projection_fusion.patch` orders projection roles, validates contiguous Q/K/V bounds, head counts, and head dimensions, and replaces materialized slices/reshapes with `nlp_create_qkv_heads_decode`. Q and K RMSNorm are recreated on the fused rank-4 results.

The QKV concept crosses TTIR and TTNN: TTIR supplies candidate ordering and role information; TTNN owns the composite operation and runtime contract. Its isolated gain is **+1.15%**. Rank-3 RoPE belongs to the foundation bundle.

4.4 KV-cache dtype and role propagation

KV-cache update operations are stateful boundaries. Their return types must carry the selected device dtype, and graph-role metadata must survive unary operations so later fusions can still identify query, key, and value paths. The relevant TT-MLIR and TT-XLA patches are primarily correctness/enabling work: they keep a low-precision, fused graph well typed and recognizable.

These patches illustrate why optimization accounting should not be reduced to "one commit, one speedup." A type-inference fix may add no direct throughput yet be required for the layout and kernel that do.

5 Layout, precision, and runtime policy

5.1 BF8 lowering and fast host packing

The server requests `bfp_bf8` for eligible weights. TT-XLA compile-option patches enable the intended single-chip lowering; TT-MLIR assigns the device dtype; TTNN and TT-Metal consume BFLOAT8_B tiles. The `fast_bfloat16_bfp8_pack.patch` concept accelerates creation of standard 32-by-32 BFP8 tiles from BF16 host weights.

These changes attack two different phases:

- BF8 storage reduces steady-state device weight traffic during decode.
- faster packing reduces model-load and preparation cost on the host.

The cumulative measurements do not isolate either effect from the foundation and pre-existing baseline. The report therefore describes their mechanism but does not invent a per-patch speedup.

5.2 Width-sharded decode RMSNorm

The default logical height of RMSNorm during batch-one decode is one. A generic height-parallel program cannot occupy many cores. The runtime patch recognizes a device-resident, tiled, unsharded input with sequence length one and width at least 2048. When the width divides into 16 tile-aligned shards, it:

1. creates an 8-by-2 row-major core grid;
2. width-shards the tensor into the cores' L1 memories;
3. derives the layernorm program configuration from that shard specification;
4. executes sharded RMSNorm; and
5. restores the requested output memory configuration.

The conversion cost is paid twice, but the width-parallel reduction and affine work more than repay it. The isolated throughput gain is **+13.61%**. This is a runtime specialization rather than a new IR operation: the compiler has already identified RMSNorm, while the runtime knows the concrete tensor and device geometry.

5.3 Decode-specific validation changes

Two small TT-Metal patches admit layouts selected by the optimized graph:

- SDPA decode accepts an L1-interleaved query rather than requiring the prior sharded form.
- layernorm accepts the single-core height-sharded case produced on the decode path.

They are deliberately narrow validation changes, not general relaxations. Their performance contribution is inseparable from the foundation bundle; their engineering value is removing avoidable conversions while retaining shape and layout checks.

6 The MLP as a vertical optimization slice

6.1 Shared-LHS gate/up projection

Qwen's two first MLP projections share an activation:

$$xW_{gate},\ xW_{up} \quad \longrightarrow \quad x[W_{gate}\ W_{up}].$$

`tt_mlir_fuse_shared_lhs_matmul_pairs.patch` changes TTIR fusion eligibility so a pair, rather than only a group of at least three, can be combined. This reduces duplicate activation reads and launches, but creates a doubled-width output that the following SwiGLU still slices and rereads. In isolation it **regresses throughput by 1.69%** (Holm-adjusted $p = 8.81 \times 10^{-5}$).

The representation is nevertheless useful: the true epilogue consumes exactly this paired result. It is an enabling IR transformation whose standalone materialization is more expensive than two separate projections.

6.2 Why consumer-side SiLU fusion is insufficient

An intermediate experiment placed SiLU as a unary activation on the following TTNN multiply. That removes one elementwise operation but retains the expensive boundary:

```
combined matmul → pack full gate/up tensor → memory
                → read/unpack → SiLU + multiply → pack result
```

The measured effect is **-0.65%**. Its raw Welch p-value is 0.0266, but the Holm-adjusted p-value is 0.0533, so the experiment does not establish a family-wise-significant regression—and provides no evidence of a speedup. It demonstrates that operation-count reduction is not the same as traffic reduction.

6.3 True Dst-resident matmul-SwiGLU epilogue

The successful concept is implemented by two patches:

- `tt_mlir_fuse_matmul_swiglu.patch` recognizes the TTNN graph, introduces the fused matmul semantic, extends the FlatBuffer representation, serializes it, and invokes the corresponding runtime path.
- `matmul_swiglu_epilogue.patch` adds TTNN validation, a program factory, and the TT-Metal reader/receiver/sender/compute kernels.

The contract is intentionally specialized: one physical tile row, BF16 activation/output, BFLOAT8_B weights, DRAM-interleaved tensors, no transpose or bias, and a grid that can supply 96 workers. For Qwen3-8B, each worker owns four gate tiles and four corresponding up tiles—exactly the eight active 16-bit Dst slots.

The program performs the following sequence:

1. two sender cores multicast activation K blocks over the NoC;
2. each of 96 workers reads its paired gate/up weight tiles;
3. the matrix engine accumulates all eight output tiles;
4. partial K results use BF16 packer-L1 accumulation;
5. final gate/up partials are loaded together into Dst;
6. SiLU transforms Dst slots 0–3;
7. each gate tile multiplies its corresponding up tile in place; and
8. only four final BF16 tiles are packed and written.

The full doubled-width gate/up tensor never exists as a TTNN tensor. This producer-side boundary contributes **+2.73%** end-to-end throughput.

6.4 Prefill coverage and fallback removal

The matcher was then generalized from decode-specific naming to any supported one-tile-row input, which includes short prefill. The older consumer-side SiLU-multiply fallback was removed; validation remains at the TTNN/TT-Metal boundary, where unsupported dtype, layout, shape, and hardware combinations fail explicitly.

The change is performance-neutral: **-0.19%, p = 0.532**. It simplifies the implementation and broadens coverage without changing the fast kernel or the observed completion hash.

6.5 SwiGLU K-blocking sweep

The initial epilogue uses `in0_block_w = 2`. Every two K tiles it packs eight partial-result tiles into L1, then resumes packer-L1 accumulation. A plausible hypothesis was that widths 4, 8, or 16 would reduce synchronization and partial traffic enough to win, at the cost of larger activation and weight circular buffers.

`matmul_swiglu_epilogue.patch` now exposes `TT_METAL_SWIGLU_IN0_BLOCK_W={2,4,8,16}` and sizes the circular buffers from the chosen value. The 32-sample test compares width 4 with width 2 while disabling the new down-projection specialization:

K block	Pure decode mean ± SD	95% CI	Change	Welch p
2 tiles	26.122 ± 0.402 tok/s	[25.977, 26.267]	—	—
4 tiles	26.192 ± 0.338 tok/s	[26.071, 26.314]	+0.27%	0.452

The confidence intervals overlap closely and the observed difference is small. There is no defensible speedup, so width two remains the default. Widths 8 and 16 are retained for controlled experiments, not selected in production. Wider blocks also change accumulation order and therefore can change greedy token sequences even when the output remains deterministic.

6.6 A 110-core fused-residual down projection

The second MLP matmul has the exact decode shape 1×12288 by 12288×4096. The generic program used 64 cores and achieved only 246.2 GB/s of effective BFP8 weight bandwidth, well below the SwiGLU kernel. `down_projection_110_core.patch` adds an exact-shape Blackhole program selected only for BF16 activation/output, BFLOAT8_B weights, a BF16 residual, DRAM-interleaved tensors, and an 11-by-10 worker grid.

The design avoids a K split and its cross-core partial reduction. It partitions the 128 output tile columns unevenly across all 110 workers:

- 18 wide workers each compute two N tiles, covering columns 0–35;
- 92 narrow workers each compute one N tile, covering columns 36–127;
- one activation sender multicasts to the other 109 workers;
- weights are read exactly once;
- K is processed four tiles at a time with packer-L1 accumulation; and
- the residual addition occurs in the final compute kernel before output.

Wide and narrow workers require distinct partial-result circular-buffer descriptors. An early version allocated a two-tile ring on narrow workers even though they produced one tile; partial accumulations alternated slots and corrupted output. Separating the one- and two-tile rings fixed the accumulation protocol. The corrected path is deterministic and produces coherent text.

The specialization is enabled by default for its exact contract. `TT_METAL_DOWN_PROJECTION_110_CORES=0` selects the generic fallback in the same binary, which makes the A/B comparison independent of compilation drift.

7 Runtime trace capture for short prefill

Decode already executes as a recorded TT-Metal trace: a stable command sequence is replayed while request-dependent buffers are refreshed. Before the prefill change, the five-token prompt—padded to one 32-row tile—was compiled but its operations were submitted separately from the host.

Setting `SGLANG_TT_TRACE_DECODE_ONLY=false` records the fixed-shape prefill sequence too. In a same-binary 32-sample A/B test:

Metric	Decode-only trace	Prefill + decode trace	Change
Time to first token	618.84 ± 4.76 ms	**59.15 ± 3.70 ms**	**-90.44% (10.46x)**
Total streaming time	5.5067 ± 0.0598 s	**4.9299 ± 0.0680 s**	**-10.48%**
Pure decode throughput	25.986 ± 0.315 tok/s	26.079 ± 0.362 tok/s	+0.36%, p = 0.278

The mechanism and data agree: about 560 ms disappears before the first token, while decode is statistically unchanged. In the primary cumulative benchmark, the full-trace configuration adds **+10.23%** end-to-end throughput for a 128-token response.

8 Packaging, build, and startup patch concepts

Not every libtt patch is a steady-state model optimization. A second class makes the single-library deployment possible or keeps compilation/startup tractable. These changes should be evaluated by buildability, artifact self-containment, load time, and cold start—not assigned fabricated decode speedups.

Table 8.1: Non-model patch groups in the current libtt build.

Concept	Representative files	Purpose
Embedded runtime assets	`tt_metal_runtime_root.cc`, `tt_metal_runtime_auto_setup.cc`, Bazel embed rules	Compress TT-Metal runtime files into `libtt.so`, extract to a fingerprinted temporary root, and initialize before PJRT.
Static integration and symbol hygiene	`BUILD.bazel`, `libtt_pjrt_exports.lds`	Link the upstream plugin and internal libraries into one shared object while exposing only PJRT ABI symbols.
Bazel overlays	`third_party/*/BUILD.overlay.bazel`, `tt_xla.BUILD.bazel`, `tt_mlir.BUILD.bazel`	Bring separately built upstream projects into one central graph with pinned sources and toolchains.
Remove unused fabric/runtime surfaces	`disable_fabric_*.patch`, `disable_inspector_runtime.patch`	Exclude components unnecessary for the single-chip libtt artifact and avoid unresolved or duplicated runtime dependencies.
Linkability and TLS constraints	`reduce_static_tls.patch`, `remove_initial_exec_tls.patch`, public-UMD include patch	Make large statically integrated components safe to load as a PJRT shared library.
Compiler/runtime trimming	TTNN-only registration/runtime patches, cold stubs, disabled TT-Lang resolver, fast sharded-module check	Avoid bringing unused compiler/runtime paths into the artifact and reduce cold compilation work.
API/version compatibility	protobuf, Shardy, affine, constructor-name, map/include patches	Reconcile pinned revisions inside one build without relying on a preinstalled matching stack.

These are architectural enablers of agentic development as much as deployment features. A single build can be less convenient if it is slow, fragile, or polluted by unused subsystems; the integration patches keep the full-stack workspace practical.

9 Benchmark methodology

9.1 Hardware and software

Item	Value
Accelerator	Tenstorrent Blackhole P150
Firmware observed at startup	19.6.0
Host CPU	Intel Core Ultra 9 185H, 22 logical CPUs online
Host OS/kernel	Linux 6.8.0-124-generic, x86-64
Model	`Qwen/Qwen3-8B`
Model/activation dtype	BF16
Weight-lowering request	`bfp_bf8`
Attention backend	TT
Serving frontend	`/home/pcmoritz/sglang-jax` at `24eb823ed97e`
Established main snapshot	`627a32d`
Current kernel snapshot	`37d5460`
Benchmark date	15 July 2026, America/Los_Angeles

The P150 exposes 120 Tensix workers in this firmware configuration [14]. The SwiGLU program uses 96; the new down projection uses 110. Results are single-device and shape-specific.

9.2 Serving command

The server uses the command documented in the repository README:

```
env -u TT_METAL_RUNTIME_ROOT \
  PYTHONPATH=/home/pcmoritz/sglang-jax/python \
  PJRT_NAMES_AND_LIBRARY_PATHS=tt:/home/pcmoritz/libtt/bazel-bin/libtt.so \
  JAX_PLATFORMS=tt \
  JAX_USE_SHARDY_PARTITIONER=false \
  JAX_COMPILATION_CACHE_DIR=/tmp/sglang-jax-qwen3-8b-jax-cache \
  SGLANG_TT_HOST_WEIGHT_LOAD=1 \
  SGLANG_TT_OPTIMIZATION_LEVEL=1 \
  SGLANG_TT_EXPERIMENTAL_WEIGHT_DTYPE=bfp_bf8 \
  SGLANG_TT_TRACE_DECODE_ONLY=false \
  /home/pcmoritz/sglang-jax/.venv/bin/python -m sgl_jax.launch_server \
  --model-path Qwen/Qwen3-8B \
  --host 127.0.0.1 --port 31000 \
  --device tt --dtype bfloat16 --attention-backend tt \
  --max-running-requests 2 --max-total-tokens 1024 \
  --max-prefill-tokens 256 --chunked-prefill-size 256 \
  --page-size 32 --watchdog-timeout 1200 \
```

```
  --disable-precompile --skip-server-warmup \
  --disable-overlap-schedule --disable-radix-cache
```

Requests are serial despite the capacity of two. Radix caching and overlap scheduling are disabled so every request executes the same prompt prefill and does not overlap another request. Optional CPU/fused sampling paths remain at their disabled defaults; this report concerns the main model path.

9.3 Request and sample policy

The primary sequence sends the same request 34 times:

```
{
  "text": "The capital of France is",
  "sampling_params": {"temperature": 0, "max_new_tokens": 128}
}
```

The first two requests are discarded because they include graph compilation, TT-Metal kernel compilation, trace setup, and cache population. The next 32 complete requests form the analysis window. Primary throughput is

$$T_j = \frac{128}{L_j},$$

where L_j is SGLang's server-reported end-to-end latency.

The current kernel experiment uses the streaming completions endpoint and 32 retained requests per configuration. Pure decode throughput is 127 inter-token intervals divided by the elapsed time from first to last token. Streaming end-to-end throughput is 128 divided by loopback-client total time. The exact per-request observations are in `data/current-kernel-samples.csv`.

9.4 Statistical treatment

Means, sample standard deviations, and two-sided 95% Student-t intervals are reported. Same-metric configurations use a two-sided Welch t-test, which does not assume equal variance. The nine planned adjacent comparisons in the older cumulative sequence use Holm's step-down correction. The two new kernel A/B tests are separately planned experiments and retain raw Welch p-values.

These statistics quantify request-to-request variation within the recorded windows. They do not remove sequential drift, autocorrelation, or shape/model dependence. Effect sizes and mechanistic profiles are more informative than a threshold alone.

10 Results

10.1 Established concept sequence

The sequence below is chronological only because incremental attribution needs a predecessor. The report's technical discussion is organized by the patch concepts above, and commit identifiers remain in the CSV provenance rather than serving as section headings.

Table 10.1: End-to-end 128-token generation, 32 retained requests per measurement stage.

Stage	Concept introduced	Mean ± SD (tok/s)	95% CI	Incremental	vs. baseline	Holm p
V0	Documented serving baseline	16.265 ± 0.128	[16.218, 16.311]	—	0.00%	—
V1	Decode foundation: semantic recovery, dtype, and layout	19.938 ± 0.283	[19.836, 20.040]	**+22.58%**	+22.58%	2.62e-44
V2	Expanded-SiLU recognition	20.596 ± 0.279	[20.495, 20.696]	**+3.30%**	+26.63%	1.09e-12
V3	QKV projection fusion	20.832 ± 0.293	[20.726, 20.938]	**+1.15%**	+28.08%	0.00481
V4	Two-way shared-LHS matmul	20.479 ± 0.321	[20.363, 20.594]	**-1.69%**	+25.91%	8.81e-5
V5	Decode RMSNorm width sharding	23.265 ± 0.235	[23.181, 23.350]	**+13.61%**	+43.04%	3.25e-42
V6	Consumer-side SiLU/multiply fusion	23.113 ± 0.296	[23.007, 23.220]	-0.65%	+42.11%	0.0533
V7	Dst-resident matmul-SwiGLU epilogue	23.744 ± 0.311	[23.632, 23.856]	**+2.73%**	+45.98%	5.90e-11
V8	Prefill-capable, fallback-free path	23.698 ± 0.274	[23.599, 23.797]	-0.19%	+45.70%	0.532

Stage	Concept introduced	Mean ± SD (tok/s)	95% CI	Incremental	vs. baseline	Holm p
V9	Fixed-shape prefill trace	**26.123 ± 0.394**	[25.981, 26.265]	**+10.23%**	**+60.61%**	5.54e-34

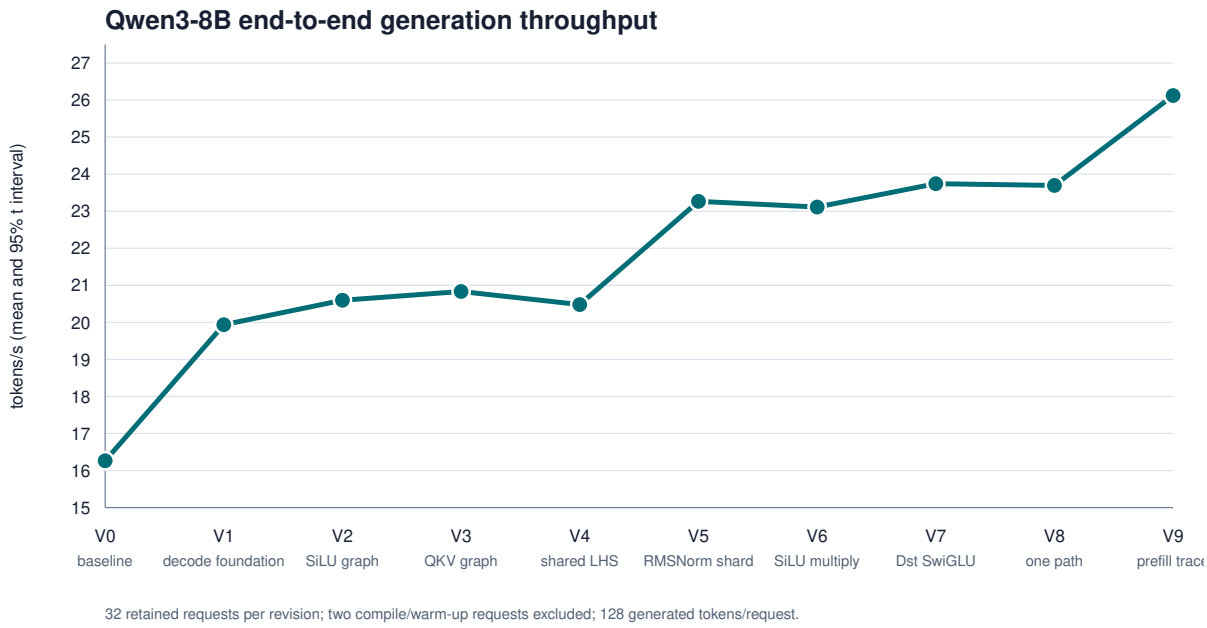

Figure 10.1: Cumulative throughput across the measured concepts.

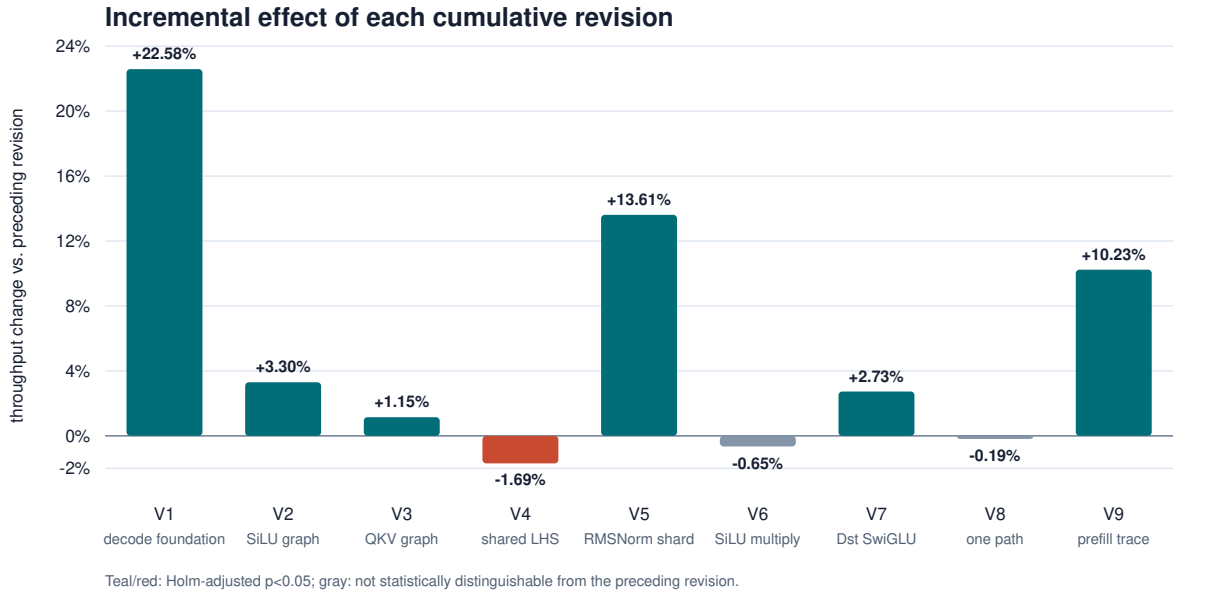

Figure 10.2: Incremental effect of each concept in the established sequence.

The negative and neutral rows are important. Shared-LHS fusion creates a useful representation but materializes it inefficiently; consumer-side SiLU fusion does not remove the producer boundary; and fallback removal changes coverage, not the kernel. A concept count that retained only successful experiments would hide the evidence that led to the final fusion boundary.

10.2 Current kernel A/B experiments

Table 10.2: Current-branch streaming results, 32 retained samples per configuration. Width 4 is compared with width 2; the down projection is compared with the same width-4 binary/configuration with the specialization disabled.

Configuration	Pure decode mean ± SD	95% CI	Streaming E2E mean ± SD	Change in decode	Welch p
SwiGLU K block 2, generic down	26.122 ± 0.402	[25.977, 26.267]	26.016 ± 0.396	—	—
SwiGLU K block 4, generic down	26.192 ± 0.338	[26.071, 26.314]	26.085 ± 0.333	+0.27%	0.452
K block 4, 110-core down	**27.753 ± 0.330**	**[27.634, 27.872]**	**27.619 ± 0.322**	**+5.96%**	**3.74e-27**

The down-projection result is both statistically and practically clear. Mean decode time falls from 38.179 to 36.032 ms/token, saving 2.147 ms/token. The streaming end-to-end comparison improves by 5.88% ($p = 3.52 \times 10^{-27}$), from 26.085 to 27.619 tokens/s. TTFT remains approximately 58 ms, as expected for a decode-kernel change.

10.3 Device profile of the down projection

The direct TT-Metal profile identifies 36 occurrences per token, matching the 36 Qwen3-8B decoder layers. Exactly those operation positions change from 64 to 110 workers.

Metric	Generic program	110-core specialization	Change
Workers	64	110	+71.9%
Mean kernel time/layer	217.188 µs	154.932 µs	**-28.66%**
Effective BFP8 weight bandwidth	246.2 GB/s	345.2 GB/s	**+40.18%**
Projected 36-layer saving	—	2.241 ms/token	—

The projected kernel saving is close to the observed 2.147 ms/token. The new program reaches about 92% of the previously measured 375.1 GB/s SwiGLU effective bandwidth. This agreement between operation profile and end-to-end measurement supports the causal attribution.

10.4 Upstream tt-inference-server reference

Tenstorrent's tt-inference-server is an OpenAI-compatible service built around vLLM and TT-Transformers [19]. It enters the stack above TTNN rather than through JAX, StableHLO, and TT-MLIR:

```
libtt:  SGLang-JAX → JAX/StableHLO → PJRT/TT-XLA/TT-MLIR → TTNN → TT-Metal
TTIS:   OpenAI API → vLLM → TT-Transformers                → TTNN → TT-Metal
```

The v0.10.0 reference uses the same P150, Qwen3-8B model, prompt, 128-token output, serial request policy, and disabled prefix cache.

Implementation	Mean ± SD (tok/s)	95% CI	Mean latency
libtt documented baseline	16.265 ± 0.128	[16.218, 16.311]	7.870 s
libtt established main	**26.123 ± 0.394**	[25.981, 26.265]	**4.901 s**
TTIS v0.10.0	24.148 ± 0.412	[23.999, 24.296]	5.302 s

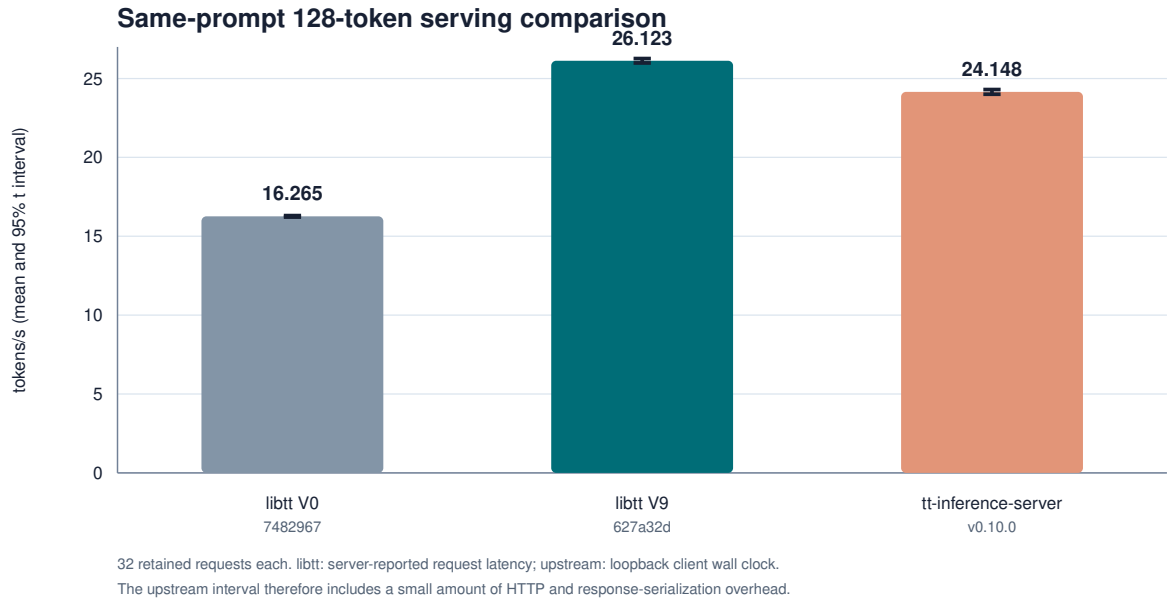

Figure 10.3: External serving-stack reference on the same model workload.

Established-main libtt is 8.18% above the recorded upstream mean. The comparison is system-level, not a kernel attribution: the model implementations, serving frontends, trace strategies, and clocks differ. In particular, TTIS is measured at the loopback client while the older libtt sequence uses request-local server time. The current streaming kernel results are not inserted into this table because their endpoint and collection window also differ.

10.5 Correctness and numeric behavior

All 32 requests within every configuration are deterministic. The established sequence changes token hash when graph algebra or reduction order changes. The current kernel experiment produces these deterministic completion-text hashes:

Configuration	SHA-256 prefix
SwiGLU block 2, generic down	`5119e79e42b5`
SwiGLU block 4, generic down	`c917933082e3`
SwiGLU block 4, 110-core down	`c041ccb1901d`

The specialized result is coherent—for example, it continues "Paris. The capital of Germany is Berlin. The capital of Italy is Rome…"—but determinism and plausibility are not a quality evaluation. BF8 arithmetic, reassociation, and reduction order can perturb logits and later greedy choices. Perplexity and task-level evaluation remain necessary before treating a changed token stream as production-equivalent.

11 Engineering interpretation

11.1 Choose the abstraction by the information it preserves

The optimization map suggests a practical rule:

- use **TTIR** when the problem is algebraic recognition or graph structure;
- use **TTNN IR** when the compiler needs a named runtime operation, layout, or device dtype;
- use the **TTNN runtime** when the choice depends on concrete tensor and device state;
- use a **TT-Metal program factory** for core topology, circular buffers, multicast, and program selection; and
- use **device kernels** for Dst lifetime, pack/unpack traffic, tile arithmetic, and synchronization.

Moving a change lower too early loses semantic information. Keeping it too high cannot control the data movement that determines decode performance.

11.2 Fusion should be described by the eliminated boundary

"Fused" is not a performance explanation. The MLP experiments distinguish three boundaries:

```
shared-LHS matmul:
  removes duplicate activation work, retains doubled output       → -1.69%

SiLU on BinaryNg input:
  removes one elementwise intermediate, retains matmul output     → -0.65%

matmul-SwiGLU epilogue:
  keeps gate/up in Dst and writes only the final half-width result → +2.73%
```

The useful description is what no longer crosses Dst, L1, DRAM, or an operation launch boundary.

11.3 Geometry can beat a more complicated reduction

The original down-projection recommendation considered a 2D K/N partition with cross-core reduction to employ more workers. Profiling showed the actual target was weight bandwidth. Uneven N partitioning uses 110 workers without rereading weights or reducing partials across cores. The solution is simpler and reaches 345.2 GB/s—close enough to the SwiGLU reference to deliver most of the expected benefit.

11.4 Negative results belong in the design record

Wider SwiGLU blocking was plausible from code inspection: fewer partial packs should reduce synchronization. The measured null result says another cost—CB footprint, overlap, or the non-dominance of those packs—offsets that saving. Keeping the knob while retaining the measured default is more useful than silently selecting the largest block.

The same principle applies to shared-LHS and consumer-side fusion. The central build makes such experiments cheap; the report makes their outcomes durable.

12 Limitations and follow-up work

1. **One model and decode regime.** Results apply to Qwen3-8B, a five-token prompt padded to one tile, serial 128-token generation, and a single P150. Larger batches, context lengths, or continuous batching can move the bottleneck.
2. **Cumulative attribution is ordered.** The established stages are real cumulative builds, not a factorial experiment. Patch effects can interact; the shared-LHS representation and SwiGLU epilogue demonstrate this directly.
3. **The foundation group is not decomposed.** Its +22.58% covers several compiler, dtype, validation, and layout changes. Isolating them would require additional compatible builds.
4. **Two metric families.** The older sequence uses server-reported request latency; the current kernel experiments use streaming client timing and a pure decode clock. Their absolute means should not be subtracted as if they came from one randomized block.
5. **Sequential sampling.** Configurations were not randomized or interleaved. Thermal and background drift can remain. Repeated randomized server blocks would strengthen publication-grade inference.
6. **Profile naming required positional alignment.** The final profiling CSV lacked operation names because of metadata capture behavior. Its 854-row structure aligned exactly with an earlier named trace, and the 36 changed positions match the down projections, but a repeated named profile would be preferable.
7. **Shape-specific kernels.** The 96-core epilogue and 110-core down projection are guarded for Blackhole and exact Qwen3-8B decode geometry. Other models, Wormhole, and multi-chip execution need different configurations.
8. **Quality is unmeasured.** Deterministic coherent completion is a smoke test, not evidence of equivalent perplexity or task accuracy.
9. **External reference scope.** The TTIS v0.10.0 image is official, but P150 model metadata was added locally because that release listed Qwen3-8B on P300 rather than P150. The runtime image itself was unchanged.

The most promising next MLP experiment is now an on-chip boundary between the SwiGLU output and down projection: either preserve a compatible sharded layout or consume the output without a DRAM round trip. Unlike the completed 110-core specialization, that change would require coordinated producer/consumer shard contracts and is therefore another full vertical slice.

13 Reproduction and report generation

The report directory contains:

- `report.md`: canonical source;
- `data/samples.csv` and `data/summary.csv`: established 32-sample sequence;
- `data/current-kernel-samples.csv` and `data/current-kernel-summary.csv`: the blocking/down-projection A/B data;
- `data/current-kernel-manifest.json`: exact current experiment provenance;
- `data/down-projection-profile-summary.csv`: per-operation profile summary;
- `data/latest-main-streaming-*`: direct prefill trace A/B data;
- `data/upstream-tt-inference-*`: TTIS observations and manifest;
- `analyze.py`: statistics and SVG generation;
- `benchmark_upstream.py`: upstream reference collector;
- `figures/*.svg`: vector figures; and
- `Makefile` and `style.css`: HTML, LaTeX, and PDF generation.

To regenerate statistics when the raw `/tmp` benchmark directories are available:

```
/home/pcmoritz/sglang-jax/.venv/bin/python \
  docs/libtt-optimization-report/analyze.py
```

To build a self-contained HTML report:

```
make -C docs/libtt-optimization-report html
```

To produce the typeset PDF with vector figures:

```
make -C docs/libtt-optimization-report pdf
```

The PDF path uses Pandoc, XeLaTeX, `booktabs`, `microtype`, controlled page geometry, numbered sections, a generated table of contents, and SVG-to-vector conversion. Generated HTML, LaTeX, and PDF artifacts are versioned with the source so the report can be reviewed without installing the toolchain.

14 Conclusion

libtt is best understood not as a thin adapter but as a deployable slice of an open accelerator stack. One `libtt.so` contains the PJRT backend, compiler, runtime, and runtime assets; one Bazel graph pins and builds them; and one patch surface reaches from StableHLO/TTIR semantics to Tensix dataflow. That structure is deliberately similar to the operational role of `libtpu.so`, while retaining the distinctive advantage that libtt's complete Tenstorrent stack can be inspected and changed.

The performance record validates that architecture. Graph recovery enables dedicated operations; layout policy exposes decode width parallelism; a cross-layer matmul-SwiGLU representation removes the correct materialization; and a program-factory/kernel specialization raises down-projection bandwidth by 40.2%. The established line improves end-to-end Qwen3-8B throughput by 60.61%. The newest down projection adds another 5.96% to pure decode in a same-build A/B test, while the blocking sweep correctly remains an experimental knob after failing to show a significant gain.

The central lesson is architectural: inference performance is a property of the path through IR, layout, runtime, data movement, and arithmetic—not of any one layer in isolation. libtt's self-contained build and open patch surface make that path a tractable unit of agentic development.

Bibliography

1. Z. Tan, B. Kang, and A. Narasimham, "A Developer's Guide to Debugging JAX on Cloud TPUs," Google Developers Blog, 2026. Describes `libtpu.so` as the shared library containing the XLA compiler, TPU driver, and hardware communication logic. https://developers.googleblog.com/a-developers-guide-to-debugging-jax-on-cloud-tpus-essential-tools-and-techniques/
2. Bazel Project, "Hermeticity." https://bazel.build/concepts/hermeticity
3. OpenXLA Project, "PJRT—Uniform Device API." https://openxla.org/xla/pjrt
4. OpenXLA Project, "StableHLO Specification." https://openxla.org/stablehlo/spec
5. C. Lattner et al., "MLIR: Scaling Compiler Infrastructure for Domain-Specific Computation," *2021 IEEE/ACM International Symposium on Code Generation and Optimization*, pp. 2–14, 2021. https://doi.org/10.1109/CGO51591.2021.9370308
6. Tenstorrent, "TT-Forge: Open-Source AI Compiler Stack." https://github.com/tenstorrent/tt-forge
7. Tenstorrent, "TT-MLIR: Architecture and Dialect Overview." https://docs.tenstorrent.com/tt-mlir/overview.html
8. Tenstorrent, "TT-MLIR Tensor Layout." https://docs.tenstorrent.com/tt-mlir/specs/tensor-layout.html
9. Tenstorrent, "TT-Metalium Getting Started and Programming Model." https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/get_started/get_started.html
10. Tenstorrent, "Compute Engines and Data Flow within Tensix." https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/advanced_topics/compute_engines_and_dataflow_within_tensix.html
11. Tenstorrent, "Circular Buffer APIs." https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/apis/kernel_apis/circular_buffers/circular_buffers.html
12. Tenstorrent, "Tiles." https://docs.tenstorrent.com/tt-metal/latest/tt-metalium/tt_metal/advanced_topics/tiles.html
13. Tenstorrent, "TTNN Tensor: Layout, Sharding, and BFLOAT8_B." https://docs.tenstorrent.com/tt-metal/latest/ttnn/ttnn/tensor.html
14. Tenstorrent, "Blackhole PCIe Card Documentation" and firmware release notes. https://docs.tenstorrent.com/tt-system-firmware/boards/tenstorrent/tt_blackhole/doc/index.html
15. SGLang Project, "SGLang-JAX." https://github.com/sgl-project/sglang-jax
16. A. Yang et al., "Qwen3 Technical Report," arXiv:2505.09388, 2025. https://arxiv.org/abs/2505.09388
17. B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," *Advances in Neural Information Processing Systems 32*, 2019. https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html
18. N. Shazeer, "GLU Variants Improve Transformer," arXiv:2002.05202, 2020. https://arxiv.org/abs/2002.05202
19. Tenstorrent, "tt-inference-server v0.10.0." https://github.com/tenstorrent/tt-inference-server/releases/tag/v0.10.0